



Lecture 3

Robustness to Functional Form Assumptions II

Dave Armstrong

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

Why might this assumption fail?

- The estimated relationship is too restrictive.
- The estimated relationship is overly complex when a simpler one is sufficient.
- The estimated relationship imposes shape constraints that are not supported by the data.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What kinds of inferential instability can result?

- Wrong relationship shape.
- Improper estimation of effect sizes/directions.
- Spurious thresholds, turning points, or saturation effects.
- Poor predictions.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What tools exist to evaluate robustness?

- Dummy coding independent variables.
- Monotone regression relationships.
- Polynomials and non-linear transformations
- Regression splines
- Smoothing-splines/Generalized Additive Models

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What are the limits of these tools?

- Non-linear transformations and polynomials require arbitrary choice of power or basis dimension.
- Dummy coding independent variables is only really possible for ordinal variables with relatively few levels.
- Regression splines can be noisy.
- Smoothing splines/GAMs require smoothing parameters to be chosen either arbitrarily or tuned from the data.

Today's Robustness Question

What assumption/abstraction are we perturbing?

- That we have estimated the appropriate functional form of the relationship between x and y .

What are the limits of these tools?

- Non-linear transformations and polynomials require arbitrary choice of power or basis dimension.
- Dummy coding independent variables is only really possible for ordinal variables with relatively few levels.
- Regression splines can be noisy.
- Smoothing splines/GAMs require smoothing parameters to be chosen either arbitrarily or tuned from the data.

Data-driven functional forms may fit the current sample well but fail to generalize to new data.

Functional Form Assumptions.

When estimating a parametric (linear) model, particularly when interested in the relationship between x and y , we estimate:

$$y_i = a + f(x_i) + f(\mathbf{z}_i) + e_i$$

Where:

- x is a variable of interest
- $\mathbf{z}$ is a vector of covariates.

Generally $f(x)$ is parametric e.g.,:

- linear: $f(x) = b_1 x$
- log: $f(x) = b_1 \log(x)$
- quadratic: $f(x) = b_1 x + b_2 x^2$

Functional Form Assumptions.

When estimating a parametric (linear) model, particularly when interested in the relationship between x and y , we estimate:

$$y_i = a + f(x_i) + f(\mathbf{z}_i) + e_i$$

Where:

- x is a variable of interest
- $\mathbf{z}$ is a vector of covariates.

Generally $f(x)$ is parametric e.g.,:

- linear: $f(x) = b_1 x$
- log: $f(x) = b_1 \log(x)$
- quadratic: $f(x) = b_1 x + b_2 x^2$

The precise choice of $f(x)$ is usually guided by theory.

Plausible Alternatives

These slides are largely focused on identifying *parametric* plausible alternatives to linearity (or any baseline functional form).

- We will use residual diagnostics to identify plausible departures from the baseline functional form.
- Some departures suggest non-linear transformations, while others suggest adding additional terms (e.g., polynomials) or moving to more flexible specifications.

Plausible Alternatives

These slides are largely focused on identifying *parametric* plausible alternatives to linearity (or any baseline functional form).

- We will use residual diagnostics to identify plausible departures from the baseline functional form.
- Some departures suggest non-linear transformations, while others suggest adding additional terms (e.g., polynomials) or moving to more flexible specifications.

Remember - robustness is about how conclusions fare in the face of *plausible* perturbations of assumptions we make, not necessarily every possible perturbation.

Non-linearity

The assumption that the average error $E(\varepsilon)$ is everywhere zero implies that the regression surface accurately reflects the dependency of Y on the X 's

- That is - we assume we've chosen the right functional form.
 - Violating this assumption implies that the model fails to account for a systematic pattern between Y and the X 's

It is impossible to directly view the regression surface when more than two predictors are specified

- We can employ *partial residual plots* to assess non-linearity.

Partial-Residual Plots (C+R plots)

For any linear model of y on x and $\mathbf{Z}$, $y_i = f(x_i) + g(\mathbf{z}_i) + e_i$ the partial residual is:

$$r_i^{(x)} = \hat{f}(x_i) + \hat{e}_i$$

- In a model linear in x , it would be $r_i^{(x)} = \hat{b}_x x_i + \hat{e}_i$
- For a model quadratic in x , it would be $r_i^{(x)} = \hat{b}_1 x_i + \hat{b}_2 x_i^2 + \hat{e}_i$

The "partial residuals" $r_i^{(x)}$ are plotted versus x .

- A non-parametric smooth (e.g., LOESS curve) helps identify the shape of the partial relationship between x and y with minimal functional form assumptions.
- If the relationship is appropriately captured, the LOESS curve should follow $\hat{f}(x)$ reasonably closely.
- Systematic departures from $\hat{f}(x)$ indicate potential unmodeled signal.

Example: The Canadian Prestige Data

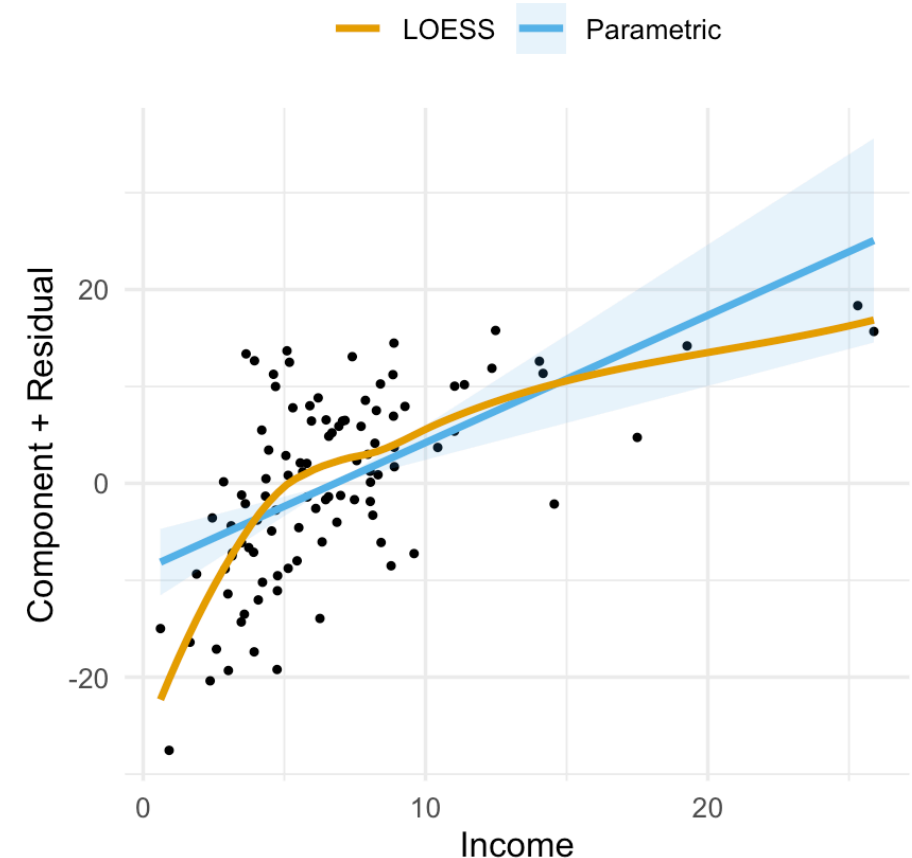
```
data(Prestige, package="carData")
Prestige$income <- Prestige$income/1000

Prestige <- Prestige |>
  drop_na(prestige, income, education, women)

pmod<-lm(prestige ~ education +
  women + income, data=Prestige)

cr_dat <- get_cr_data(pmod, data = Prestige, term="income")

ggplot(cr_dat, aes(x=x)) +
  geom_point(aes(y = cr), size=.75) +
  geom_ribbon(aes(ymin = lwr, ymax=upr,
    fill = "Parametric"),
    alpha=.15, color="transparent") +
  geom_line(aes(y = f_hat, color="Parametric")) +
  geom_smooth(aes(y=cr, color="LOESS",
    fill="LOESS"),
    method="loess",
    se=FALSE, span=.75) +
  scale_color_okabe_ito() +
  scale_fill_okabe_ito() +
  labs(x="Income", y="Component + Residual",
    fill="", color = "") +
  theme_minimal() +
  theme(legend.position = "top")
```



LOESS

LOESS (Local Regression) works in the following way:

1. Pick a value of x at the lower edge of the distribution (e.g., the minimum) and call this x_0 .
2. Identify the *span* percent of points closes to x_0 on x .
3. Estimate a weighted linear or polynomial regression of y on x , weighting points closer to x_0 higher than points farther from x_0 .
4. Generate a prediction and from that model at x_0 and mark that as $\hat{y}_0$.
5. Move x_0 to the right a little bit and repeat until you reach $\max(x)$.
6. Connect the predictions.

The *span* $(0,1]$ controls the tradeoff between smoothness and flexibility.

- Larger spans $\rightarrow$ smoother curves
- Smaller spans $\rightarrow$ more flexible ("wigglier") curves



Two Dimensions of Nonlinearity

To identify plausible alternatives, we need to know *what kind* of unmodeled non-linearity exists. There are two relevant evaluation criteria:

Two Dimensions of Nonlinearity

To identify plausible alternatives, we need to know *what kind* of unmodeled non-linearity exists. There are two relevant evaluation criteria:

Simple vs. Complex

- Simple means the curvature of the function relating x to y does not change direction (i.e., there is no inflection point).
- Complex means that there is an inflection point.

Monotone vs Non-monotone

- Monotone means that as x increases the function relating x to y never decreases or x increases the function relating x to y never increases, depending on the nature of the function.

Handling Non-linearity: Common Strategies

Simple, monotone

- Transformations of Y and/or X

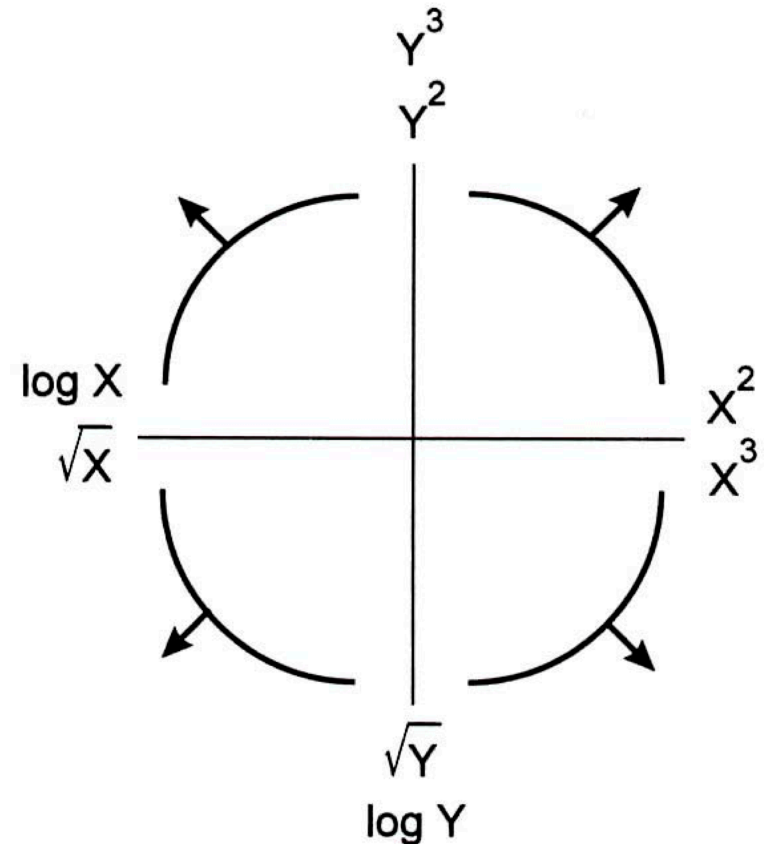
Complex Non-linearity

- Polynomial Regression
- If pattern has too many turns, polynomials tend to oversmooth peaks
- Regression Splines
- More complicated non-parametric models.

Identifying Monotone Alternatives

- The direction of the bulge indicates the appropriate type of power transformation for Y and/or X
- A bulge to the top left of the scatterplot suggests transforming Y up the ladder of powers and/or X down the ladder of powers will straighten the relationship

Figure 4.6 from Fox (1997)



Ad hoc Transformations

Common transformations such as $\log(x)$, $\sqrt{x}$, or x^2 are often useful starting points when investigating potential non-linearity.

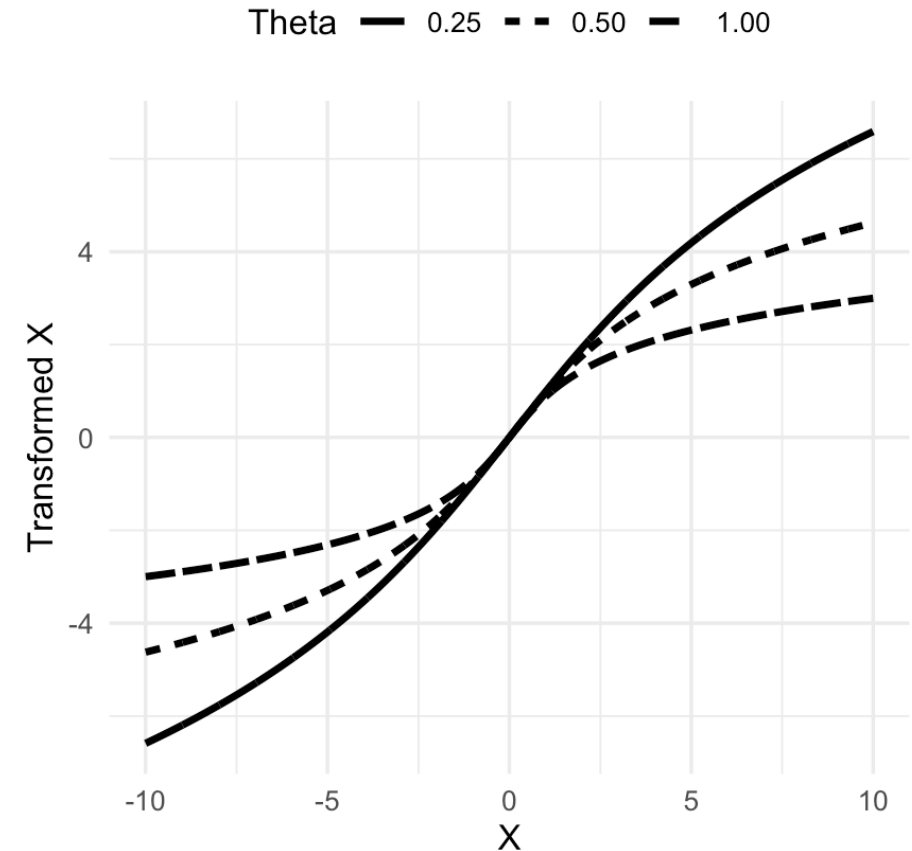
- They correspond to familiar relationship shapes (e.g., diminishing returns, accelerating effects, or curvature).
- Many substantive theories imply one of these forms, either explicitly or implicitly.
- Because these transformations are widely used, their implications are generally easy to interpret and communicate.
- They provide a simple benchmark before moving to more flexible alternatives.

Some transformations (like $\log(x)$) will require x to be either non-negative or strictly positive.

Inverse Hyperbolic Sine Transformation

Sometimes, the log transform is not the most useful because a variable has lots of zeros and you don't want to add an arbitrary constant to all counts. The IHS transformation is a good alternative.

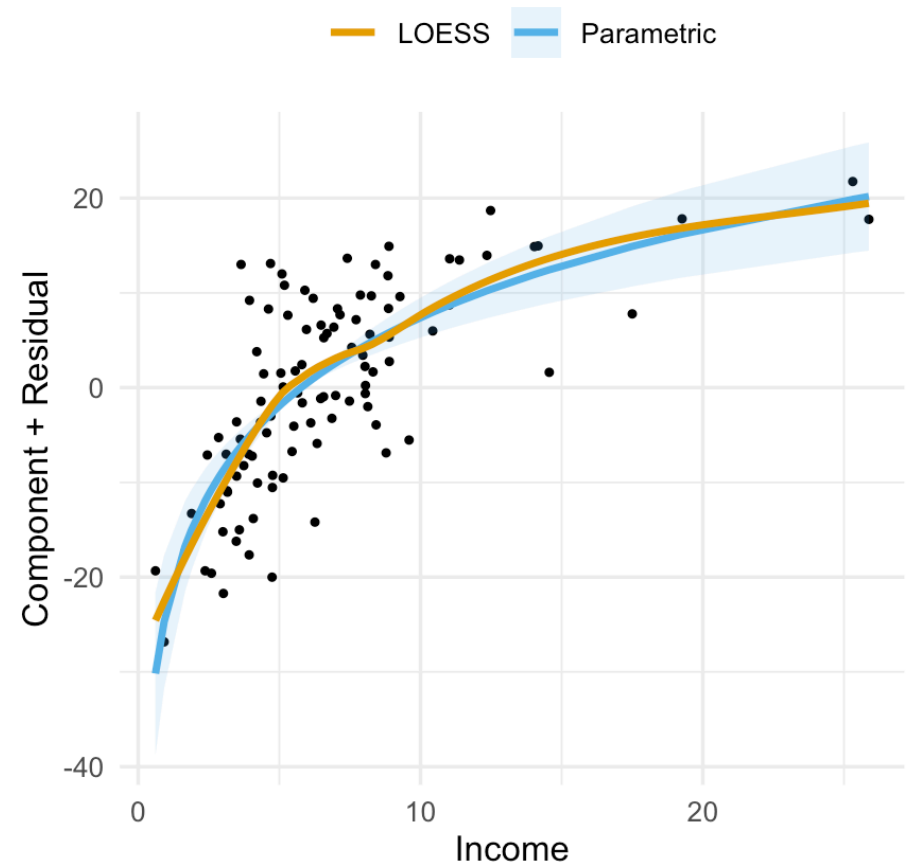
$$\begin{aligned} \text{IHS}(x) &= \frac{\sinh^{-1}(\theta x)}{\theta} \\ &= \frac{\log\left(\theta x + \log(\theta x^2 + 1)^{\left(\frac{1}{2}\right)}\right)}{\theta} \end{aligned}$$



Income Transformation

The `income` variable appears to require a transformation down the ladder - we'll try the `log`.

```
logmod <- lm(prestige ~ education +  
  women + log(income), data=Prestige)  
log_cr_dat <- get_cr_data(logmod, data = Prestige,  
  term="log(income)")  
ggplot(log_cr_dat, aes(x=x)) +  
  geom_point(aes(y = cr), size=.75) +  
  geom_ribbon(aes(ymin = lwr, ymax=upr,  
    fill = "Parametric"),  
    alpha=.15, color="transparent") +  
  geom_line(aes(y = f_hat, color="Parametric")) +  
  geom_smooth(aes(y=cr, color="LOESS",  
    fill="LOESS"),  
    method="loess",  
    se=FALSE, span=.75) +  
  scale_color_okabe_ito() +  
  scale_fill_okabe_ito() +  
  labs(x="Income", y="Component + Residual",  
    fill="", color = "") +  
  theme_minimal() +  
  theme(legend.position = "top")
```



The LOESS curve follows the log relationship really closely, we've identified a plausible alternative.

Robustness to the Log

```
library(marginaleffects)
linpred <- predictions(pmod, variables =
  list(income = quantile(Prestige$income,
    probs=seq(0,1,by=.1))),
  by = "income")
monpred <- predictions(logmod, variables =
  list(income = quantile(Prestige$income,
    probs=seq(0,1,by=.1))),
  by="income")
se_linpred <- sqrt(diag(vcov(linpred)))
se_monpred <- sqrt(diag(vcov(monpred)))

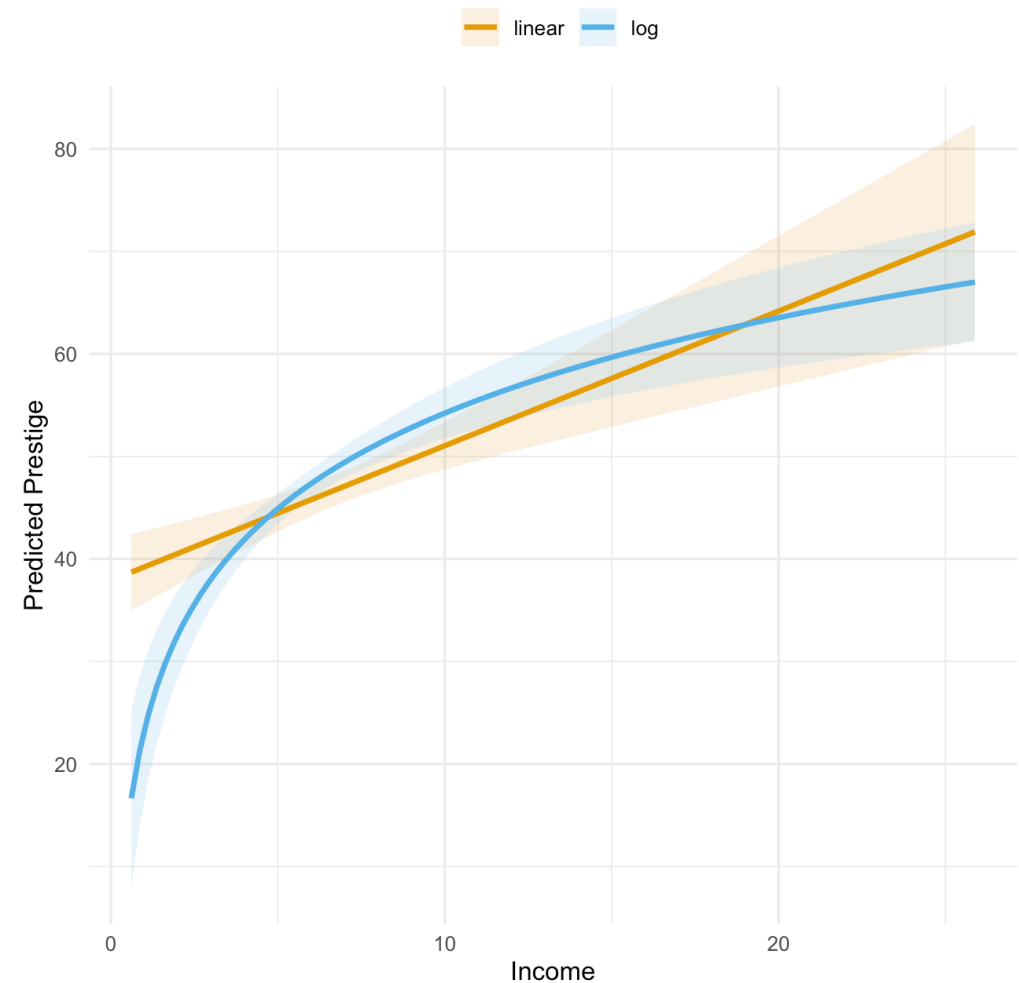
rob_log <- sapply(1:nrow(linpred), \(i)
  robustness(dnorm, dnorm,
    target_args = list(mean = coef(linpred)[i],
      sd = se_linpred[i]),
    alt_args = list(mean = coef(monpred)[i],
      sd = se_monpred[i])))
colnames(rob_log) <- paste0("p", seq(0,1,by=.1)*100)
```

```
## # A tibble: 11 × 5
##   percentile   ovl    np    js    kl
##   <chr>      <dbl> <dbl> <dbl> <dbl>
## 1 p0         0      0    0.032 0
## 2 p10        0.172 0.206 0.698 3.43
## 3 p20        0.434 0.681 0.341 1.31
## 4 p30        0.831 0.968 0.036 0.123
## 5 p40        0.698 0.941 0.105 0.396
## 6 p50        0.319 0.549 0.482 2.31
## 7 p60        0.155 0.19  0.725 4.09
## 8 p70        0.115 0.109 0.791 4.50
## 9 p80        0.13  0.132 0.766 4.11
## 10 p90       0.267 0.393 0.556 2.44
## 11 p100      0.51  0.971 0.266 1.92
```

Robustness is quite low at small values of income and again from the 60th to 80th percentiles. This is due to where the linear and non-linear curves intersect (see next slide for effects plot).

Effects

```
seq_range <- function(x, n=100){
  seq(min(x, na.rm=TRUE),
      max(x, na.rm=TRUE),
      length=n)
}
inc_seq <- seq_range(Prestige$income)
lp <- predictions(pmod, by="income",
  newdata = datagrid(income = inc_seq))
mp <- predictions(logmod, by="income",
  newdata = datagrid(income = inc_seq))
lp |> as_tibble() |>
  mutate(form = "linear") |>
  bind_rows(mp |> as_tibble() |>
    mutate(form = "log")) |>
  ggplot(aes(x=income, y=estimate,
    ymin = conf.low, ymax=conf.high,
    fill=form, color=form)) +
  geom_ribbon(alpha=.15, color="transparent") +
  geom_line() +
  theme_minimal() +
  scale_fill_okabe_ito() +
  scale_color_okabe_ito() +
  labs(x="Income", y="Predicted Prestige",
    fill = "", color="") +
  theme(legend.position = "top")
```



Box-Tidwell transformation Example: Prestige Data

Sometimes, an ad hoc search may not produce a sufficiently plausible alternative. In that case, the Box-Tidwell model may work. It estimates the transformation parameters on the indicated variables.

```
library(car)
boxTidwell(prestige ~ income ,
           ~ education + women, data=Prestige)

## MLE of lambda Score Statistic (t) Pr(>|t|)
##      0.08073      -4.8338 5.021e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## iterations = 10
```

- The statistically significant test suggests that the linear specification may be overly restrictive and that a transformed version of income is a plausible robustness alternative.
- The MLE of Power suggests that income should be transformed by a power of 0.08 (suggesting the log might work well)

Comparing Candidate Transformations

The estimated power is approximately 0.08. The log transformation provides a very similar shape and is considerably easier to interpret.

- If the candidate transformation (e.g., the log instead of 0.08) is good enough, then the transformation powers on the new data should be insignificant.

```
boxTidwell(prestige ~ I(log(income) + 1),  
           ~ education + women, data=Prestige)
```

```
## MLE of lambda Score Statistic (t) Pr(>|t|)  
##      1.2727      0.9093    0.3655  
##  
## iterations = 5
```

Notice that the p-value is > 0.05

- The theoretically familiar log transformation approximates the estimated power transformation very well.

Polynomial Regression

For non-linearity that is not simple and monotone, polynomials are a parametric alternative that may work.

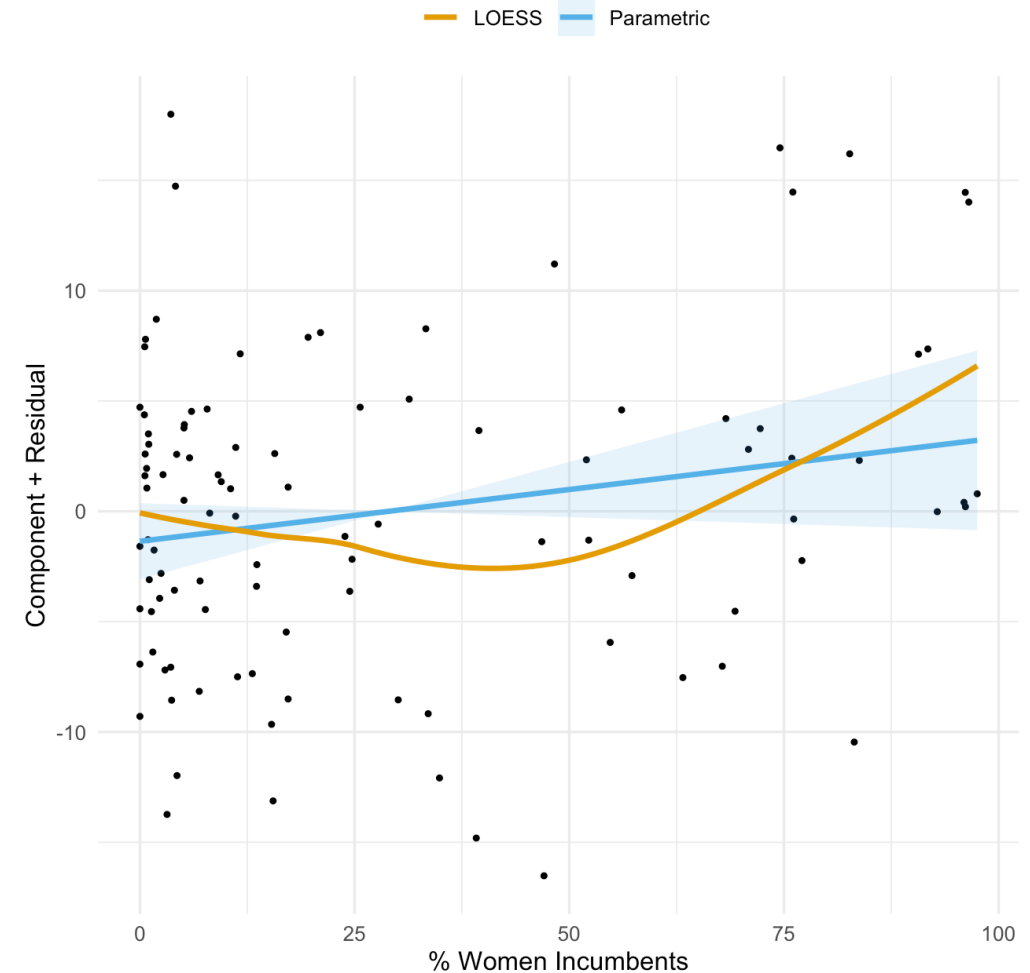
Two or more regressors of ascending power (i.e., linear, quadratic and cubic terms) are used to capture the effects of a single variable

- For every bend in the curve, we add another term to the model, going up in power each time. A polynomial of order k can have at most $k - 1$ turning points.
- Polynomial models are linear in the parameters even though they are non-linear in the variables

Order	Equation
First	$Y = \alpha + \beta_1 X$
Second	$Y = \alpha + \beta_1 X + \beta_2 X^2$
Third	$Y = \alpha + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$

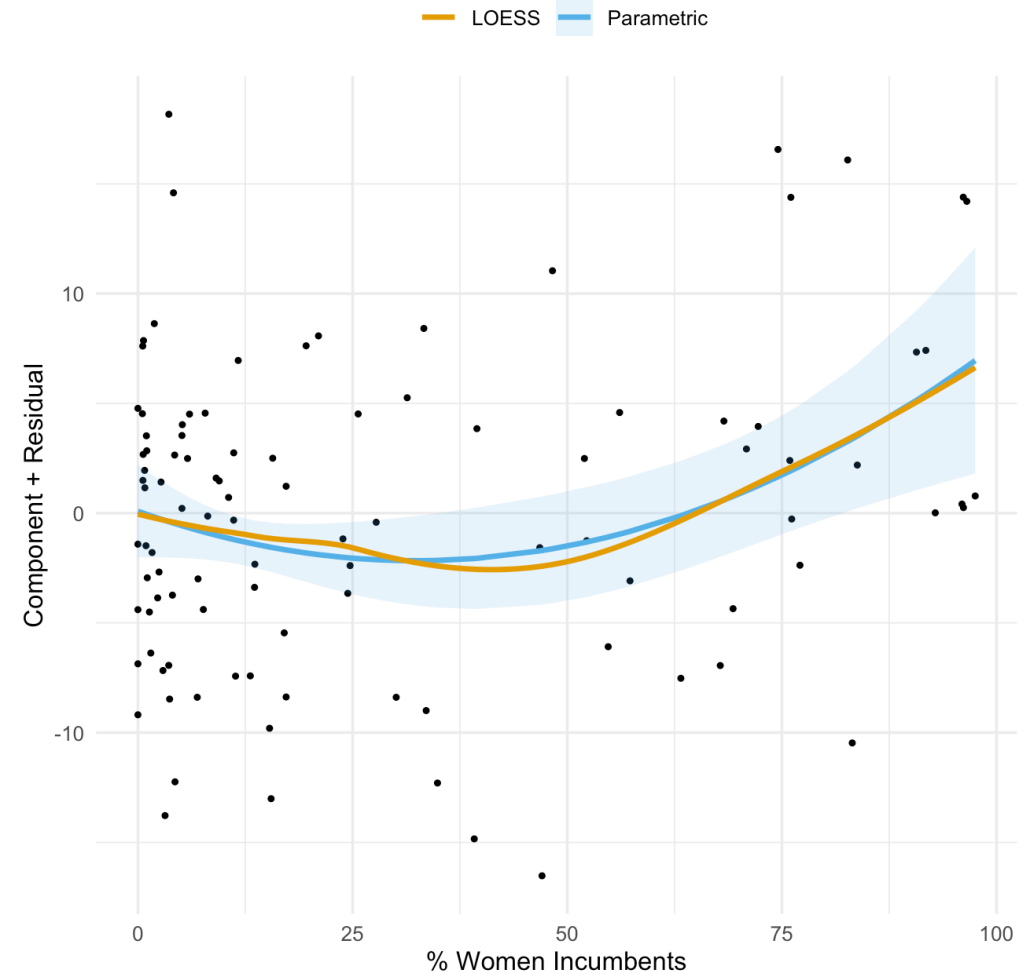
CR Plot for Women

```
cr_dat <- get_cr_data(logmod, data = Prestige,  
                      term="women")  
  
ggplot(cr_dat, aes(x=x)) +  
  geom_point(aes(y = cr), size=.75) +  
  geom_ribbon(aes(ymin = lwr, ymax=upr,  
                fill = "Parametric"),  
            alpha=.15, color="transparent") +  
  geom_line(aes(y = f_hat, color="Parametric")) +  
  geom_smooth(aes(y=cr, color="LOESS",  
                 fill="LOESS"),  
             method="loess",  
             se=FALSE, span=.75) +  
  scale_color_okabe_ito() +  
  scale_fill_okabe_ito() +  
  labs(x="% Women Incumbents", y="Component + Residual",  
       fill="", color = "") +  
  theme_minimal() +  
  theme(legend.position = "top")
```



Polynomial Model

```
polymod <- lm(prestige ~ education + log(income) +  
  poly(women, 2, raw=TRUE) , data=Prestige)  
  
poly_cr_dat <- get_cr_data(polymod, data = Prestige,  
  term="poly(women, 2, raw = TRUE)")  
ggplot(poly_cr_dat, aes(x=x)) +  
  geom_point(aes(y = cr), size=.75) +  
  geom_ribbon(aes(ymin = lwr, ymax=upr,  
    fill = "Parametric"),  
    alpha=.15, color="transparent") +  
  geom_line(aes(y = f_hat, color="Parametric")) +  
  geom_smooth(aes(y=cr, color="LOESS",  
    fill="LOESS"),  
    method="loess",  
    se=FALSE, span=.75) +  
  scale_color_okabe_ito() +  
  scale_fill_okabe_ito() +  
  labs(x="% Women Incumbents", y="Component + Residual",  
    fill="", color = "") +  
  theme_minimal() +  
  theme(legend.position = "top")
```



The LOESS curve follows the quadratic polynomial closely, we found a plausible alternative.



Robustness

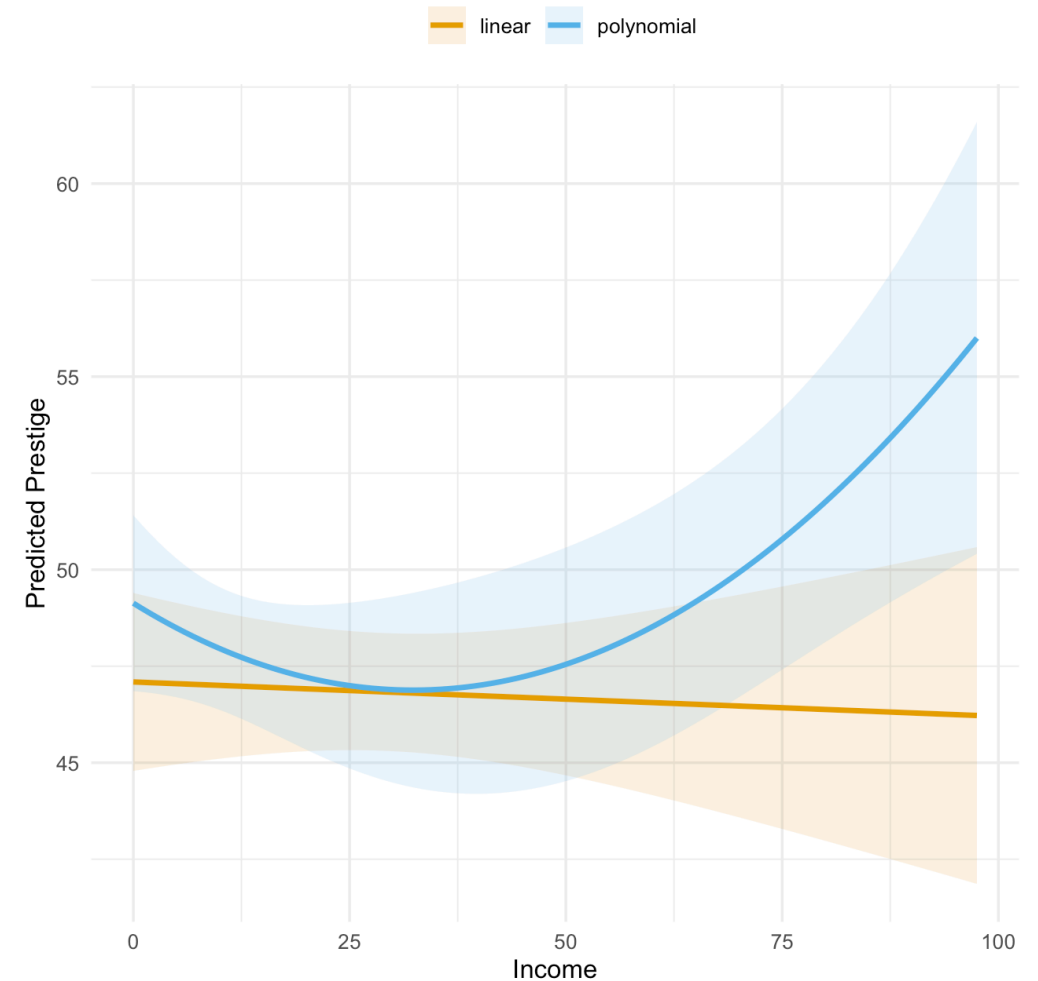
```
linpred <- predictions(pmod, variables =  
  list(women = quantile(Prestige$women,  
                        probs=seq(0,1,by=.1))),  
  by = "women")  
polypred <- predictions(polymod, variables =  
  list(women = quantile(Prestige$women,  
                        probs=seq(0,1,by=.1))),  
  by="women")  
se_linpred <- sqrt(diag(vcov(linpred)))  
se_polypred <- sqrt(diag(vcov(polypred)))  
rob_poly <- sapply(1:nrow(linpred), \(i)  
  robustness(dnorm, dnorm,  
    target_args = list(mean = coef(linpred)[i],  
                      sd = se_linpred[i]),  
    alt_args = list(mean = coef(polypred)[i],  
                   sd = se_polypred[i])))  
colnames(rob_poly) <- paste0("p", seq(0,1,by=.1)*100)
```

```
## # A tibble: 11 × 5  
##   percentile  ovl    np    js    kl  
##   <chr>      <dbl> <dbl> <dbl> <dbl>  
## 1 p0        0.936 0.929 0.005 0.014  
## 2 p10       0.906 0.931 0.01  0.028  
## 3 p20       0.837 0.928 0.03  0.084  
## 4 p30       0.744 0.911 0.073 0.222  
## 5 p40       0.577 0.834 0.195 0.681  
## 6 p50       0.386 0.604 0.398 1.56  
## 7 p60       0.26  0.312 0.562 1.98  
## 8 p70       0.353 0.402 0.432 1.23  
## 9 p80       0.795 0.908 0.047 0.131  
## 10 p90      0.303 0.475 0.505 2.21  
## 11 p100     0.123 0.116 0.777 4.01
```

Robustness is reasonably high throughout - it is worse at middle values where the quadratic curve swoops down.

Effects

```
wom_seq <- seq_range(Prestige$women)
lp <- predictions(pmod, by="women",
  newdata = datagrid(women = wom_seq))
pp <- predictions(polymod, by="women",
  newdata = datagrid(women = wom_seq))
lp |> as_tibble() |>
  mutate(form = "linear") |>
  bind_rows(pp |> as_tibble() |>
    mutate(form = "polynomial")) |>
  ggplot(aes(x=women, y=estimate,
    ymin = conf.low, ymax=conf.high,
    fill=form, color=form)) +
  geom_ribbon(alpha=.15, color="transparent") +
  geom_line() +
  theme_minimal() +
  scale_fill_okabe_ito() +
  scale_color_okabe_ito() +
  labs(x="Income", y="Predicted Prestige",
    fill = "", color="") +
  theme(legend.position = "top")
```



Conclusion: Women Incumbents

Linear in Women

```
## # A tibble: 1 × 5
##   term      estimate std.error statistic p.value
##   <chr>      <dbl>    <dbl>    <dbl>   <dbl>
## 1 women -0.00891    0.0304    -0.293   0.770
```

Quadratic in Women

```
## # A tibble: 2 × 5
##   term                                estimate std.error statistic p.value
##   <chr>                                <dbl>    <dbl>    <dbl>   <dbl>
## 1 poly(women, 2, raw = TRUE)1 -0.139    0.0868    -1.60   0.112
## 2 poly(women, 2, raw = TRUE)2  0.00215  0.000944    2.28   0.0250
```

The substantive conclusion is somewhat sensitive to the functional-form assumption. While the predicted values remain fairly similar across specifications, the inference about whether the effect is distinguishable from zero changes.

N&P Alternative

Neumayer and Plümer suggest a semi-parametric functional form test continuous variables - break them into categories and treat them as categorical.

```
Prestige <- Prestige |>
  mutate(educ_fac = ntile(education, 5),
         educ_fac = factor(educ_fac))

if_mod <- lm(prestige ~ educ_fac +
  poly(women, 2, raw=TRUE) + log(income), data=Prestige)
tidy(if_mod) |>
  filter(str_detect(term, "educ"))
```

```
## # A tibble: 4 × 5
##   term      estimate std.error statistic  p.value
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl>
## 1 educ_fac2    3.52      2.30      1.53 1.29e- 1
## 2 educ_fac3    7.73      2.40      3.23 1.73e- 3
## 3 educ_fac4   16.9      2.74      6.18 1.66e- 8
## 4 educ_fac5   27.1      2.89      9.35 4.49e-15
```

The unrestricted model is perfectly consistent with monotonicity already, so the monotonic and unconstrained models will be identical.

- The only potential violation worth considering is the equal-spacing violation.

```
ep_preds <- predictions(pmod,
  newdata = datagrid(education = Prestige$education,
                     grid_type = "mean_or_mode"),
  by="education")
ef_preds <- predictions(if_mod, variables = "educ_fac",
  by="educ_fac")
ef_map <- Prestige |>
  select(education, educ_fac) |>
  distinct() |> as_tibble()

e_preds <- ep_preds |> as_tibble() |>
  full_join(ef_map) |>
  select(education, educ_fac, base = estimate,
        base_se = std.error) |>
  left_join(ef_preds |>
    select(alt = estimate,
          alt_se = std.error,
          educ_fac))
```

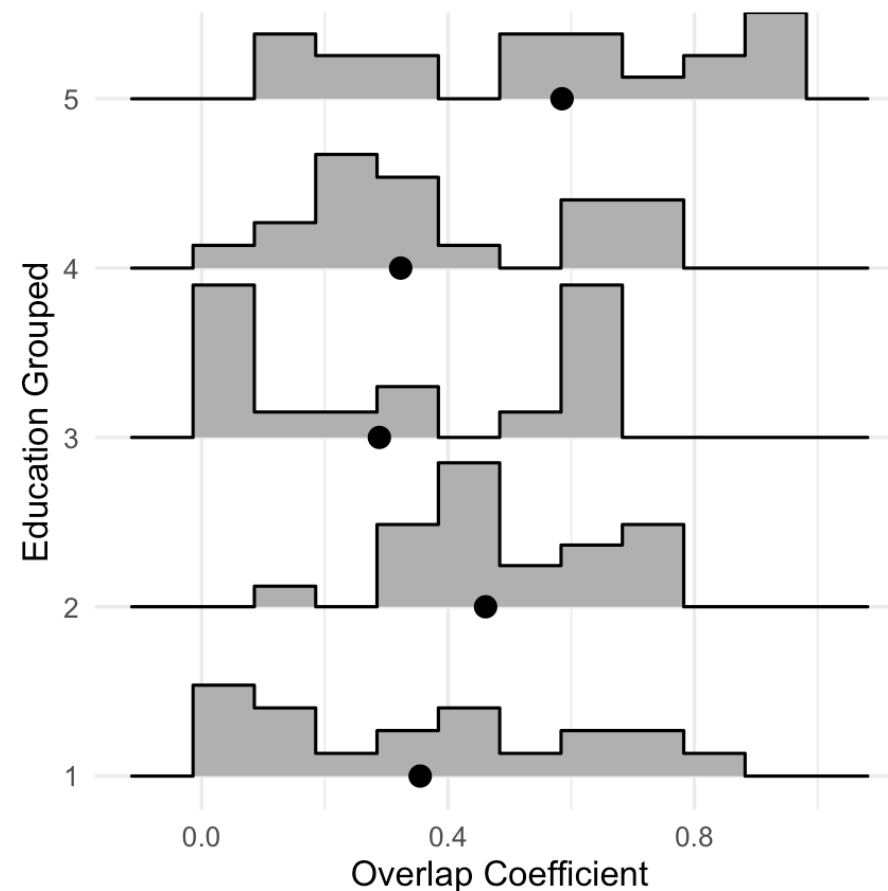

Robustness Summary

```
rob_ed <- t(sapply(1:nrow(e_preds), \(i){
  robustness(dnorm, dnorm,
    target_args = list(mean = e_preds$base[i],
                        sd = e_preds$base_se[i]),
    alt_args = list(mean = e_preds$alt[i],
                    sd = e_preds$alt_se[i]))
}))

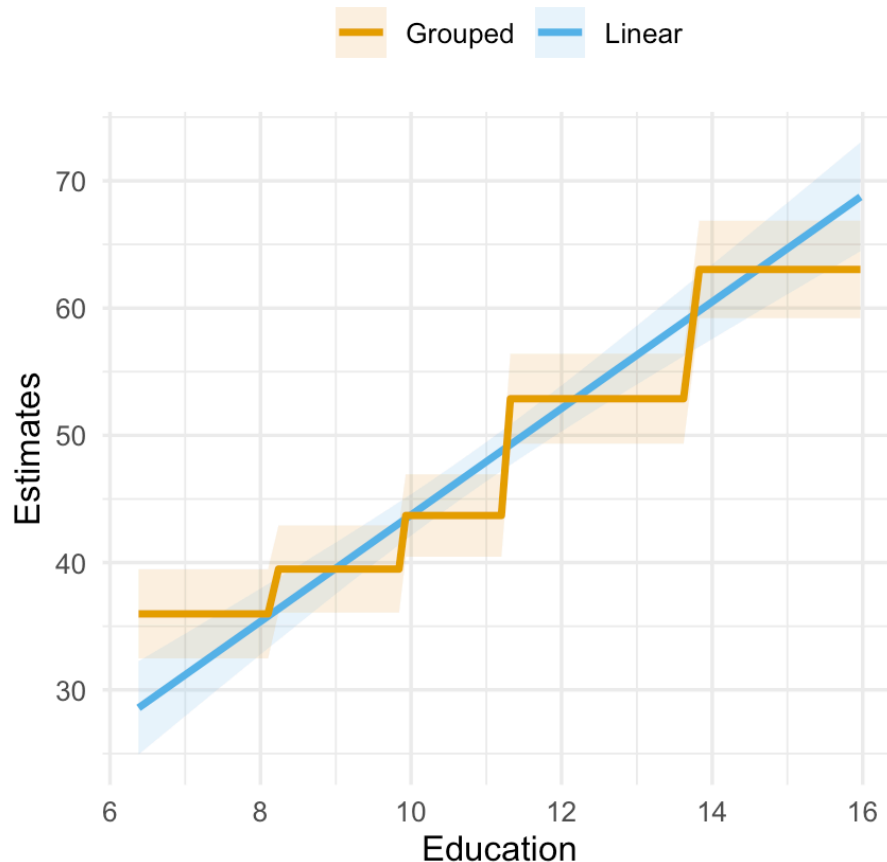
e_preds <- bind_cols(e_preds, rob_ed)

e_preds |>
  summarise(across(ovl:kl, ~median(.x)),
    .by=educ_fac)
```

```
## # A tibble: 5 × 5
##   educ_fac  ovl    np    js    kl
##   <fct>    <dbl> <dbl> <dbl> <dbl>
## 1 1      0.355 0.482 0.435 1.47
## 2 2      0.461 0.544 0.309 0.830
## 3 3      0.289 0.284 0.511 1.44
## 4 4      0.323 0.322 0.467 1.28
## 5 5      0.585 0.743 0.189 0.535
```



Effects

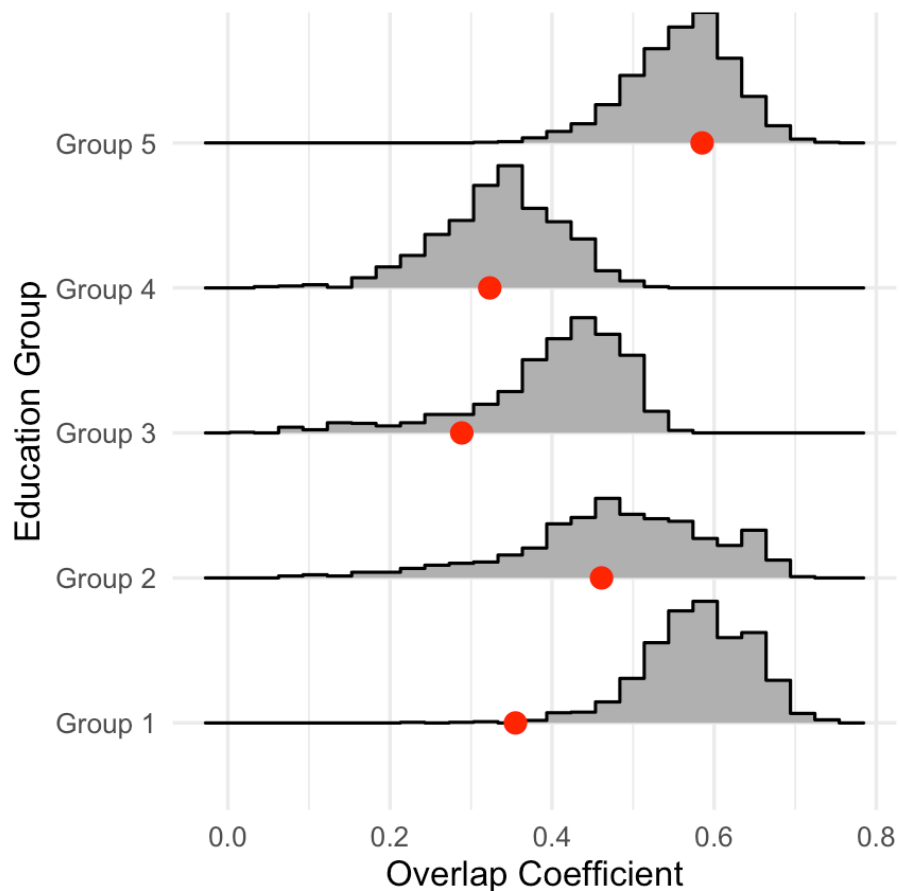


The third and fourth categories have lower robustness for different reasons

- the third tends to systematically underpredict the linear model value
- the fourth is sufficiently wide, that even though the middle of the bin tends to be about the same as the linear result, the ends of the bin are quite a ways off.

Because the grouped model approximates a continuous linear relationship using a step function, some loss of robustness is expected even when the linear specification is correct. The simulation establishes a benchmark for how much robustness we would expect under the linear model itself.

Expectations Simulation



Histograms give the simulated distribution of robustness under the linear specification. Red points are the observed robustness scores. 1

The simulation shows that groups 1 and 3 have robustness values that are quite low relative to what we might expect if the linear functional form for education was correct.

- Robustness for the other groups is around where we would expect.

See the code file for the simulation code.

Exercise

Using the Berry and Kleinberg data, estimate their first model and evaluate the robustness of the `L_hse_usnum` linear effect variable.

```
ctrls <- c("L_hse_sanction", "L_growth", "L_lnpercap", "L_lnpop", "L_polity2",  
          "L_durable", "L_civintensity", "L_spending", "L_s_us", "L_lnustrade",  
          "L_us_distance", "L_usstock2000_adj", "allfdi2000")  
f2 <- reformulate(c("L_ab_sum", "L_hse_usnum", ctrls),  
                 response="usfdi2000_adj")  
  
bkdat <- rio::import("../data/berry_klineberg_2015.dta")  
bkdat <- bkdat |>  
  drop_na(all_of(c(ctrls, "L_ab_sum", "L_hse_usnum", "usfdi2000_adj")))  
  
bkmod <- lm(f2, data=bkdat)
```

Extending the Workflow to GLMs

The same workflow applies:

1. Identify plausible alternatives.
2. Estimate alternative models.
3. Compare substantive quantities.
4. Evaluate robustness.

The only difference is that diagnostics and quantities of interest must be adapted to the GLM setting.

Partial Residuals in GLMs

For a linear model:

$$y_i = f(x_i) + g(\mathbf{z}_i) + e_i \quad \text{with} \quad r_i^{(x)} = \hat{f}(x_i) + \hat{e}_i$$

Partial Residuals in GLMs

For a linear model:

$$y_i = f(x_i) + g(\mathbf{z}_i) + e_i \quad \text{with} \quad r_i^{(x)} = \hat{f}(x_i) + \hat{e}_i$$

For a GLM:

$$g(\mu_i) = f(x_i) + h(\mathbf{z}_i)$$

the residuals are no longer additive on the response scale.

Instead, we work on the linear predictor (link) scale

$$\eta_i = g(\mu_i)$$

So, $\hat{f}(x_i)$ is x 's contribution to the linear predictor on the link scale.

Working Residuals

GLMs are estimated iteratively by repeatedly fitting approximate linear models.

The working residuals - $r_i^{(w)}$ are the residuals from these local linear approximations.

$$r_i^{(x)} = \hat{f}(x_i) + r_i^{(w)}$$

Workflow after this is identical

- C+R plots tend to be noisier and may have unusual looking patterns, but they will still highlight unmodeled structure in the same way.

Box-Tidwell for GLMs

We can estimate the Box-Tidwell transformation in a GLM. There is a function `glm_bt()` in the `nprobustness` package that does this. The formula must contain a variable raised to the `lambda` power, e.g. `I(x^lambda)`. Here's what the prestige example would look like:

```
glm_bt(prestige ~ I(income^lambda) + women + education,  
       data = Prestige, width=6)
```

```
## Box-Tidwell Power Transformation  
## -----  
## estimate    lower    upper  
##  0.08073 -0.20712  0.36743
```

Exercise

In Fearon and Laitin's widely-cited 2003 *APSR* piece, they predict the civil war onsets with a number of variables.

```
flrat <- rio::import("../data/fl_repdata.dta")
flrat$onset <- ifelse(flrar$onset > 1, 1, flrat$onset)
base_mod <- glm(onset ~ warl + gdpenl + lpopl1 + lmtnest + ncontig + Oil +
               nwstate + instab + polity2l + ethfrac + relfrac,
               data=flrat, family=binomial)
```

Thinking about `polity2l` as a continuous variable, evaluate its



Functional Form Robustness Workflow

Numeric Variables

1. Estimate baseline model.
2. Use C+R plot to identify unmodeled non-linearity.
 - If simple & monotone, non-linear transformations should be a plausible alternative.
 - If not simple & monotone, polynomials are a better place to find plausible alternatives.
3. Evaluate robustness using predictions.
4. Describe the alternative relationships when robustness is low.

The plausible alternative model will identify which assumptions are violated - constant marginal effect and/or monotonicity.